

MODULE 3: Native Capabilities

Lecture1,2: Camera, Audio, Bluetooth:

Android camera app example by camera intent

Activity_main.xml

Drag one imageview and one button from the palette, now the xml file will look like this:

File: activity_main.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity" >
```

```
<Button
```

```
    android:id="@+id/button1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true"
    android:layout_centerHorizontal="true"
    android:text="Take a Photo" >
```

```
</Button>
```

```
<ImageView
```

```
    android:id="@+id/imageView1"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:layout_above="@+id/button1"
    android:layout_alignParentTop="true"
    android:src="@drawable/ic_launcher" >
```

```
</ImageView>
```

```
</RelativeLayout>
```

File: MainActivity.java

```
package com.example.simplecamera;
```

```
import android.app.Activity;
```

```

import android.content.Intent;
import android.graphics.Bitmap;
import android.os.Bundle;
import android.view.Menu;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;

public class MainActivity extends Activity {
    private static final int CAMERA_REQUEST = 1888;
    ImageView imageView;

    public void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        imageView = (ImageView) this.findViewById(R.id.imageView1);
        Button photoButton = (Button) this.findViewById(R.id.button1);

        photoButton.setOnClickListener(new View.OnClickListener() {

            @Override
            public void onClick(View v) {
                Intent cameraIntent = new Intent(android.provider.MediaStore.ACTION_IMAGE_CAPTURE);
                startActivityForResult(cameraIntent, CAMERA_REQUEST);
            }
        });

        protected void onActivityResult(int requestCode, int resultCode, Intent data) {
            if (requestCode == CAMERA_REQUEST) {
                Bitmap photo = (Bitmap) data.getExtras().get("data");
                imageView.setImageBitmap(photo);
            }
        }
    }
}

```

```

@Override
public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu; this adds items to the action bar if it is present.
    getMenuInflater().inflate(R.menu.activity_main, menu);
    return true;
}
}

```

Bluetooth:

Bluetooth is a way to exchange data with other devices wirelessly. Android provides Bluetooth API to perform several tasks such as:

- scan bluetooth devices
- connect and transfer data from and to other devices
- manage multiple connections etc.

BluetoothAdapter class

By the help of BluetoothAdapter class, we can perform fundamental tasks such as initiate device discovery, query a list of paired (bonded) devices, create a BluetoothServerSocket instance to listen for connection requests etc.

Constants of BluetoothAdapter class

BluetoothAdapter class provides many constants. Some of them are as follows:

- String ACTION_REQUEST_ENABLE
- String ACTION_REQUEST_DISCOVERABLE
- String ACTION_DISCOVERY_STARTED
- String ACTION_DISCOVERY_FINISHED

activity_main.xml

activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="10dp"
    android:paddingRight="10dp">
<Button

```

```

        android:id="@+id/btnOn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Turn
On" android:layout_marginLeft="100dp" android:layout_marginTop="200dp" />
<Button
    android:id="@+id/btnOFF"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignBottom="@+id/btnOn"
    android:layout_toRightOf="@+id/btnOn"
    android:text="Turn OFF" />
</RelativeLayout>

```

Provide Permission

You need to provide following permissions in AndroidManifest.xml file.

1. **<uses-permission** android:name="android.permission.BLUETOOTH" />
2. **<uses-permission** android:name="android.permission.BLUETOOTH_ADMIN" />

MainActivity.java

```

import android.bluetooth.BluetoothAdapter;
import android.content.Intent;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button btntOn = (Button)findViewById(R.id.btnOn);
        Button btntOff = (Button)findViewById(R.id.btnOFF);
        final BluetoothAdapter bAdapter = BluetoothAdapter.getDefaultAdapter();
        btntOn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {

```

```

        if(bAdapter == null)
        {
            Toast.makeText(getApplicationContext(),"Bluetooth Not
Supported",Toast.LENGTH_SHORT).show();
        }
        else{
            if(!bAdapter.isEnabled()){

startActivityResult(new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE),1);
                Toast.makeText(getApplicationContext(),"Bluetooth Turned
ON",Toast.LENGTH_SHORT).show();
            }
        }
    }
});
btntOff.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        bAdapter.disable();
        Toast.makeText(getApplicationContext(),"Bluetooth Turned OFF",
Toast.LENGTH_SHORT).show();
    }
});
}
}

```

Lecture3: Sensors -how sensors work, listening to sensor readings

Android Sensors can be used to monitor the three-dimensional device movement or change in the environment of the device such as light, proximity, rotation, movements, magnetic fields, and much more.

The Android platform supports three broad categories of sensors:

Motion Sensors: These sensors measure acceleration forces and rotational forces along three axes. This category includes accelerometers, gravity sensors, gyroscopes, and rotational vector sensors.

Position Sensors: These sensors measure the physical position of a device. This category includes orientation sensors and magnetometers.

Environmental Sensors: These sensors measure various environmental parameters, such as ambient air temperature and pressure, illumination, and humidity. This category includes barometers, photometers, and thermometers.

List of Sensors

Android SDK provides you with a list of various types of sensors that you can use in your app. The availability of these sensors may vary from device to device.

Here's a quick rundown of each sensor:

- **TYPE_ACCELEROMETER**
Type: Hardware
Computes the acceleration in m/s^2 applied on all three axes (x, y and z), including the force of gravity.
- **TYPE_AMBIENT_TEMPERATURE**
Type: Hardware
Monitors the temperature of the surroundings in degrees Celsius.
- **TYPE_GRAVITY**
Type: Software or Hardware
Computes the gravitational force in m/s^2 applied on all three axes (x, y and z).
- **TYPE_GYROSCOPE**
Type: Hardware
Computes the rate of rotation in rad/s around each of the three axes (x, y and z).
- **TYPE_LIGHT**
Type: Hardware
Evaluates the light around a surrounding in lx units.
- **TYPE_LINEAR_ACCELERATION**
Type: Software or Hardware
Computes the acceleration force in m/s^2 applied on all three axes (x, y and z), excluding the force of gravity.
- **TYPE_MAGNETIC_FIELD**
Type: Hardware
Computes the geomagnetic field for all three axes in tesla (μT).
- **TYPE_ORIENTATION**
Type: Software
Computes the degree of rotation around all three axes.
- **TYPE_PRESSURE**
Type: Hardware
Computes the air pressure in hPa or mbar.
- **TYPE_PROXIMITY**
Type: Hardware
Computes the proximity of the device's screen to an object in centimeters.
- **TYPE_RELATIVE_HUMIDITY**
Type: Hardware
Computes the humidity of the surrounding air as a percentage (%).
- **TYPE_ROTATION_VECTOR**

Type: Software or Hardware

Computes the orientation of a device by the device's rotation vector.

- **TYPE_TEMPERATURE**

Type: Hardware

Monitors the temperature of the surroundings in degrees Celsius. In API 14, the **TYPE_AMBIENT_TEMPERATURE** sensor replaced this sensor.

Lecture4-5: Task 1. List the available sensors

In this task, you build a simple app that queries the sensor manager for the list of sensors available on the device.

1.1 Build the app

1. Create a new Android project. Call it SensorSurvey and use the Empty activity template.
2. Open `res/layout/activity_main.xml`.
3. Add a margin of 16 dp to the constraint layout.
4. `android:layout_margin="16dp"`
5. Delete the existing TextView.
6. Add a ScrollView element inside the constraint layout. Give it these attributes:

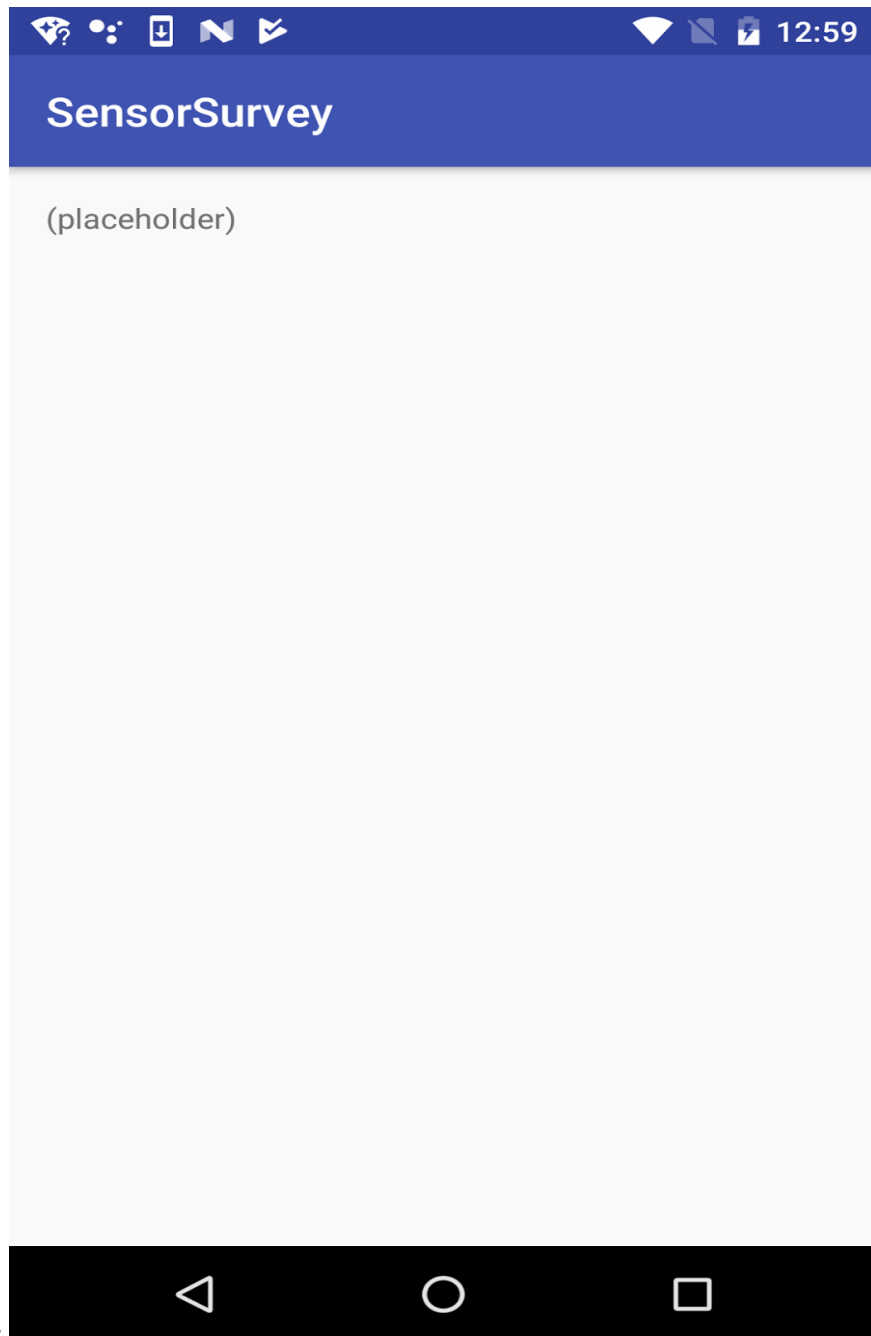
Attribute	Value
<code>android:layout_width</code>	<code>"match_parent"</code>
<code>android:layout_height</code>	<code>"match_parent"</code>
<code>app:layout_constraintBottom_toBottomOf</code>	<code>"parent"</code>
<code>app:layout_constraintTop_toTopOf</code>	<code>"parent"</code>
<code>app:layout_constraintLeft_toLeftOf</code>	<code>"parent"</code>
<code>app:layout_constraintRight_toRightOf</code>	<code>"parent"</code>

7.

Attribute	Value
-----------	-------
8.

<code>android:id</code>	<code>"@+id/sensor_list"</code>
<code>android:layout_width</code>	<code>"wrap_content"</code>
<code>android:layout_height</code>	<code>"wrap_content"</code>
<code>android:text</code>	<code>"(placeholder)"</code>

9. This TextView holds the list of sensors. The placeholder text is replaced at runtime by the actual sensor list. The layout for your app should look like this screenshot:



10.

11. Open MainActivity and add a variable at the top of the class to hold an instance of SensorManager:

12. `private SensorManager mSensorManager;`

The sensor manager is a system service that lets you access the device sensors.

13. In the onCreate() method, below the setContentView() method, get an instance of the sensor manager from system services, and assign it to the mSensorManager variable:

14. `mSensorManager =`

15. `(SensorManager) getSystemService(Context.SENSOR_SERVICE);`

16. Get the list of all sensors from the sensor manager. Store the list in a List object whose values are of type Sensor:

17. `List<Sensor> sensorList =`

18. `mSensorManager.getSensorList(Sensor.TYPE_ALL);`

The `Sensor` class represents an individual sensor and defines constants for the available sensor types. The `Sensor.TYPE_ALL` constant indicates all the available sensors.

19. Iterate over the list of sensors. For each sensor, get that sensor's official name with the `getName()` method, and append that name to the `sensorText` string. Each line of the sensor list is separated by the value of the `line.separator` property, typically a newline character:

```
20. StringBuilder sensorText = new StringBuilder();
```

```
21.
```

```
22. for (Sensor currentSensor : sensorList ) {
```

```
23.     sensorText.append(currentSensor.getName()).append(
```

```
24.         System.getProperty("line.separator"));
```

```
25. }
```

26. Get a reference to the `TextView` for the sensor list, and update the text of that view with the string containing the list of sensors:

```
27. TextView sensorTextView = (TextView) findViewById(R.id.sensor_list);
```

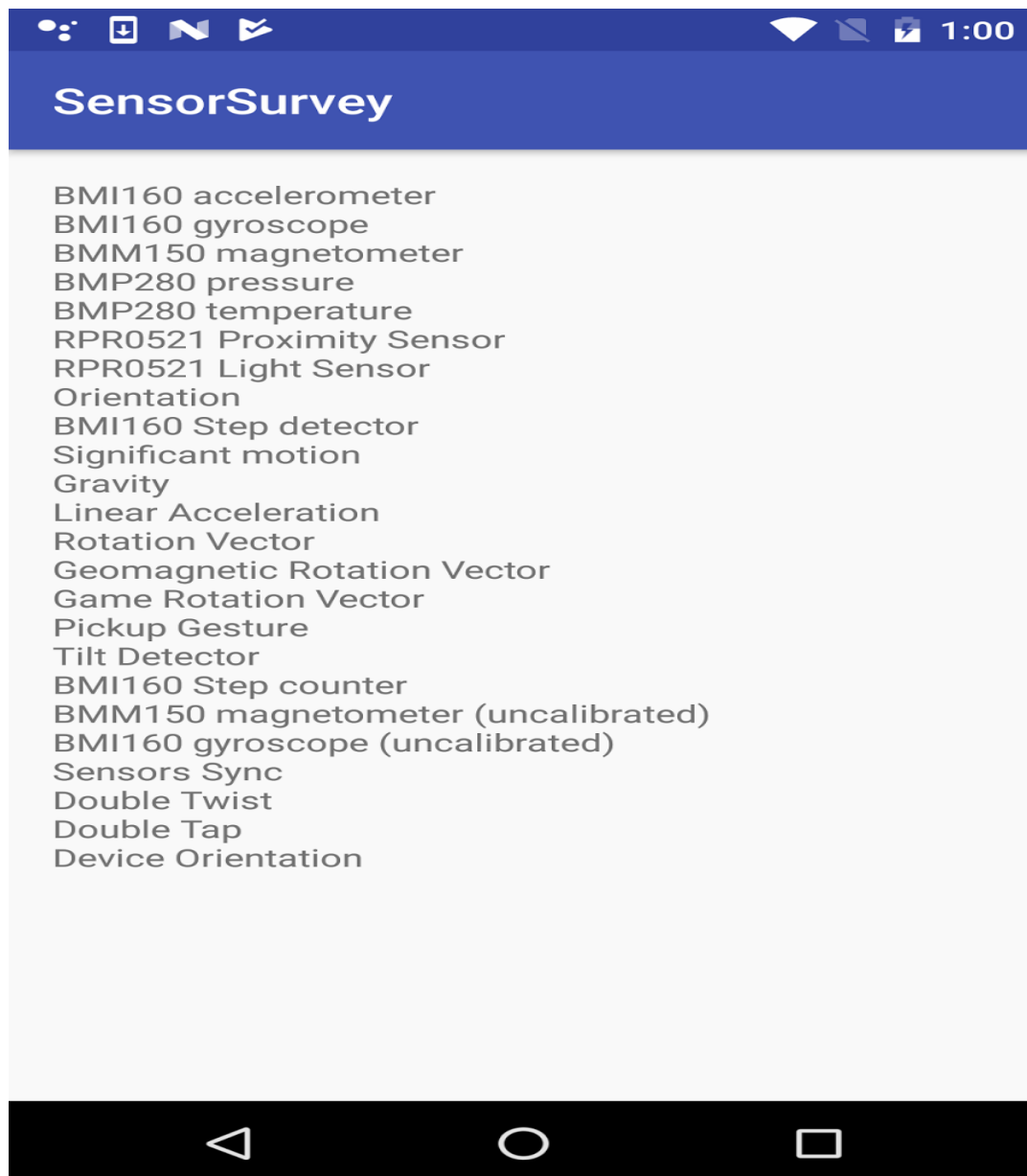
```
28. sensorTextView.setText(sensorText);
```

1.2 Run the app on a device and in the emulator

Different Android devices have different sensors available, which means the `SensorSurvey` app shows different results for each device. In addition, the Android emulator includes a small set of simulated sensors.

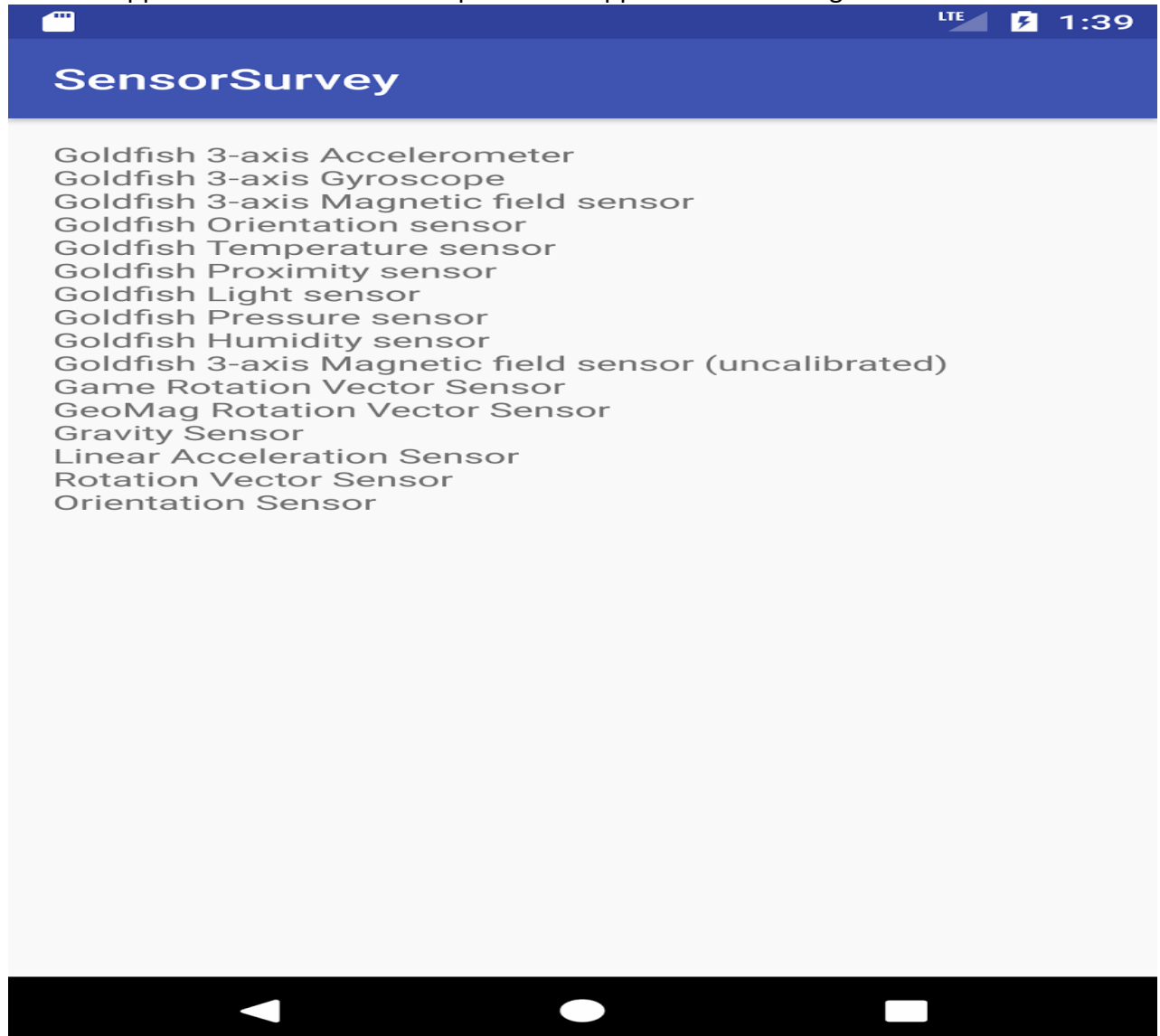
1. Run the app on a physical device. The output of the app looks something like this

screenshot:



In this list, lines that begin with a letter/number code represent physical hardware in the device. The letters and numbers indicate sensor manufacturers and model numbers. In most devices the accelerometer, gyroscope, and magnetometer are physical sensors. Lines without letter/number codes are virtual or composite sensors, that is, sensors that are simulated in software. These sensors use the data from one or more physical sensors. So, for example, the gravity sensor may use data from the accelerometer, gyroscope, and magnetometer to provide the direction and magnitude of gravity in the device's coordinate system.

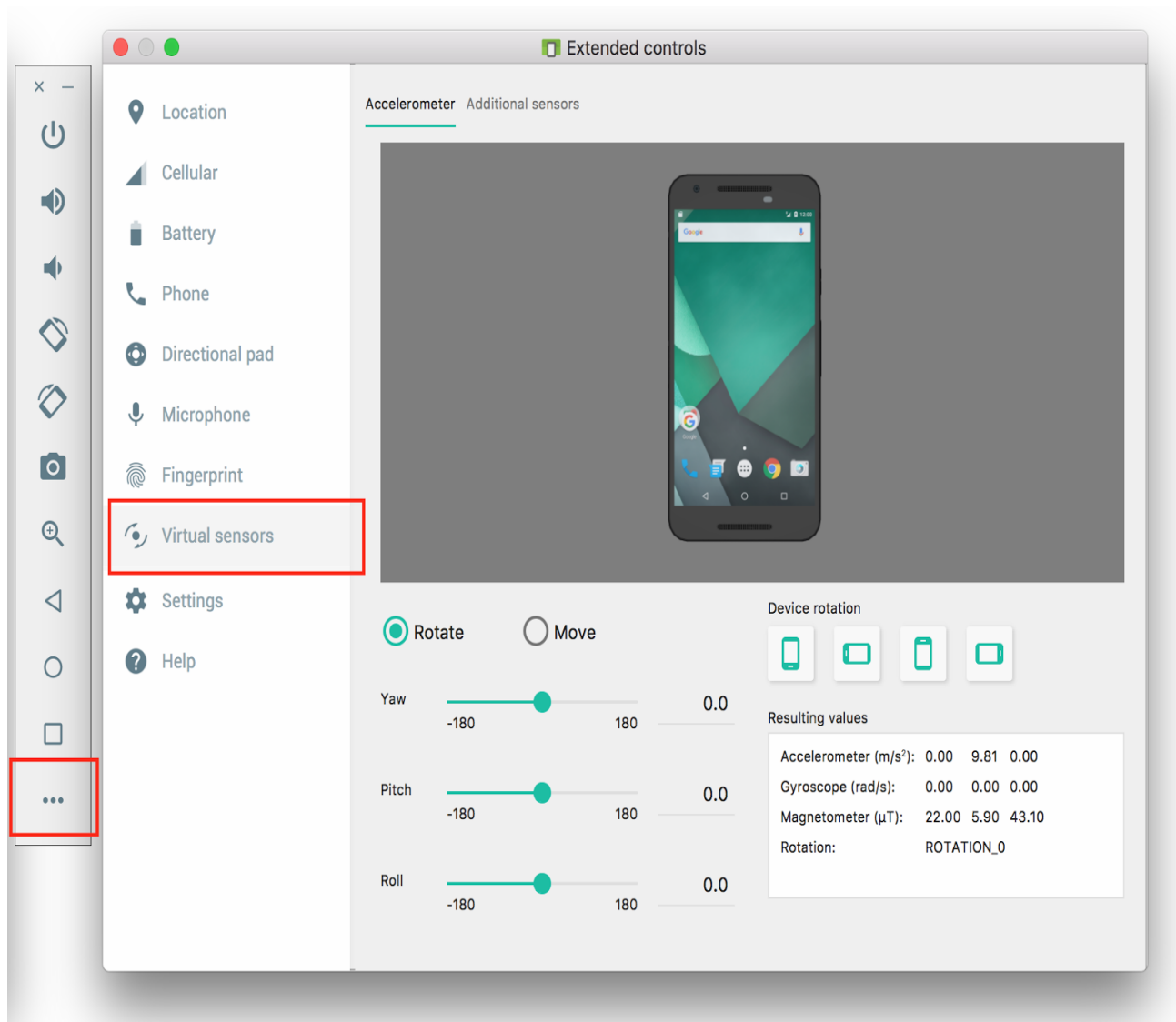
2. Run the app in an emulator. The output of the app looks something like this screenshot:



Because the Android emulator is a simulated device, all the available sensors are virtual sensors. "Goldfish" is the name of the emulator's Linux kernel.

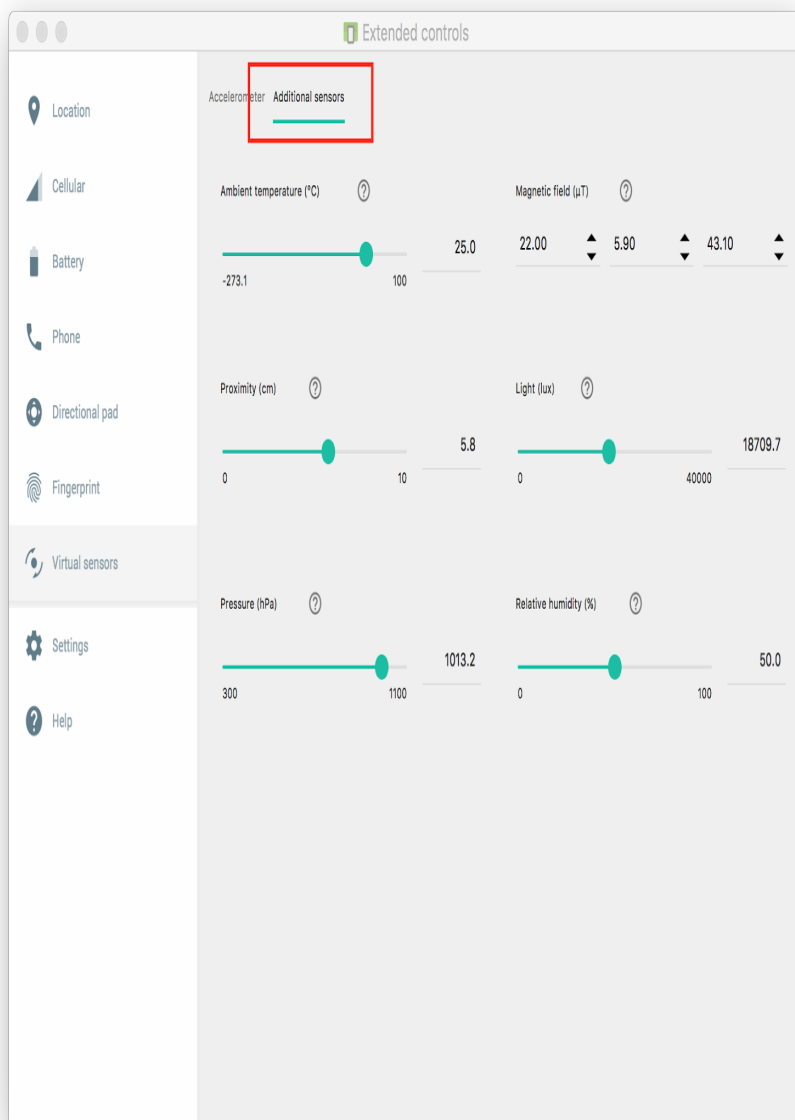
3. Click the **More** button (three horizontal dots) on the emulator's control panel. The **Extended Controls** window appears.
4. Click **Virtual Sensors**.

5.



This window shows the settings and current values for the emulator's virtual sensors. Drag the image of the device to simulate motion and acceleration with the accelerometer. Dragging the device image may also rotate the main emulator window.

6. Click the **Additional Sensors** tab.



7.

This tab shows the other available virtual sensors for the emulator, including the light, temperature, and proximity sensors. You use more of these sensors in the next task.

Lecture 6: Maps & Location: Android Communications: GPS

What is a G.P.S.?

The G.P.S. abbreviation of ***Global Positioning System is a satellite-based navigation system*** created with the help of at least 24 satellites which is expanded into six earth-centred orbits, four satellites in each orbit. No matter where you are or what weather it is, a global positioning system works 24 hours a day without any subscription or purchase, and there is no money charged for G.P.S. service.

G.P.S. helps us know where we are going or the velocity or time. Whereas we only need three satellites to know the location, the fourth satellite is used to know the location of all those three satellites and the 4th satellite automatically moves when the previous three satellites move but in 3-dimensions so that we can calculate the exact altitude of the device in which G.P.S. is used.

The United States government is the original owner of the global Positioning Systems. **The key significant players in developing the GPS technology are Ivan Getting, Bradford Parkinson & Roger Easton.** The G.P.S. Service does not ask for any data or need any data transactions; it works over simple internet or only by telephonic connections. G.P.S. also provides timely information to the users. Sometimes the areas like mountains or hills may have weak G.P.S. services as only geological positions can be identified here.

How G.P.S. works?

The Global Positioning System is circling the earth twice a day in a fixed orbit daily. There are different satellites in which every satellite creates an exclusive signal that permits the global positioning system devices so that they can decode the signal and tell the accurate locations of that satellite.

That's how the receiver of the G.P.S. gets an idea. Then by calculating the time taken by the satellite to receive a signal, it measures the same from different satellites to find the exact location of the user. Later it displays it to electronic devices as we live in a three-dimensional world, so the signal generated by each satellite is a sphere, not a circle. This is how it helps us measure our daily routines, get the map, help us find ways or find a home also helps us for adventures.

To calculate our location in 2 dimensions, A GPS takes the help of 2 or 4 satellites. If you want to know your locations in 3 dimensions, eight or more satellites are used. But depending on device, place of location and time, a G.P.S. can even work from your wrist watch (electronic G.P.S. supporting wrist bands). After knowing your location, A GPS device may also track the following things:

A little history about the GPS

The U.S. Department of Defense firstly created the G.P.S. to help the Army in 1973. But in 1980, they permit civilians to use this service after, in 1983 a huge mistake in identifying the position was made up by Korean Airways. It caused a huge accident in which nearby 269 passengers was killed, including Larry MC Donald, a United States representative.

But in 1990, the service of the Global Positioning System was weakened by the U.S. president, as converting it into "selective Ability", but later in 2000, then-President Bill Clinton signed a bill which discontinued this.

In 1999, the Indian Army faced a huge Kargil war in which having position access to the enemy locations was proven as a con. So after that, different nations started working to provide their own positioning systems. There are many G.P.S. services in different countries such as the Russian Global Navigation system (now added in G.P.S.), China's BeiDou Navigation System, European Galileo positioning system, India's NavIC and Japan's Quasi-Zenith Satellite System (QZSS) etc.

Lecture 7-8: Working with Location Manager:

This class provides access to the device's location services. These services allow the applications to obtain periodic updates of the device's geographical location, or to be notified when the device enters the proximity of a given geographical location. This class helps to leverage the location capabilities of the user's Android device.

To use this class, we first need to obtain some permissions from the user. These are:

- Manifest.permission.ACCESS_COARSE_LOCATION: This allows the app to access the approximate location of the user.
- Manifest.permission.ACCESS_FINE_LOCATION: This allows the app to access the exact location of the user.

Note that declaring the ACCESS_FINE_LOCATION permission implies the ACCESS_COARSE_LOCATION permission already.

Instantiating the Location Manager

To get a reference of the LocationManager, we use the getSystemService() method, to access the Location Service of the device.

```
LocationManager locationManager =  
    (LocationManager) this.getSystemService(Context.LOCATION_SERVICE);
```

Setting Up LocationListener

After getting the reference of the LocationManager, now we need to get the current location of the user. To do that we set up the LocationListener – an Event Listener that tells us when the location has changed. This interface has three methods:

- abstract void onLocationChanged(Location location): This method is called when the location has changed. The location object contains all the attributes of the user's current location, like Latitude, Longitude and Altitude.

- default void onProviderDisabled(String provider): This method is called when the Location Provider (or the location service of the device like GPS) has been disabled by the user.
- default void onProviderEnabled(String provider): This method is called when the Location Provider (or the location service of the device like GPS) has been enabled by the user.

Here is how we set up the LocationListener:

```
LocationListener locationListener = new LocationListener() {
@Override
public void onLocationChanged(android.location.Location location) {
//get the attributes of current location using the location variable
//example, location.getLatitude()
}

@Override
public void onProviderEnabled(String provider) {

}

@Override
public void onProviderDisabled(String provider) {

}
};
```

To get the location of the user, we use the requestLocationUpdates() method of LocationManager class. It takes in 4 arguments:

- String provider: a location provider, to see a list of all location providers, use getAllProviders(). This value cannot be null.
- long minTimeMs: minimum time interval between location updates in milliseconds. The location updates will be sent when the time specified by the parameter has passed.
- float minDistanceM: minimum distance between location updates in meters. The location will be updated when the difference between the present location and previous location is greater than or equal to this value.
- LocationListener listener: the listener to receive location updates This value cannot be null.

How to use it in code:

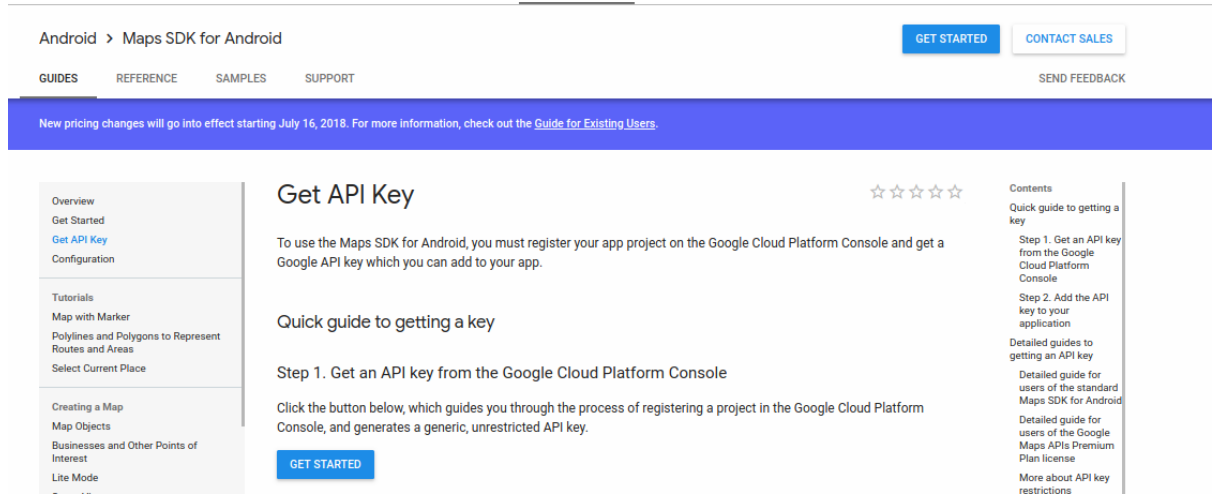
```
locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, 0, 0,
locationListener);
```

Lecture 9 Working with Google Maps extensions:

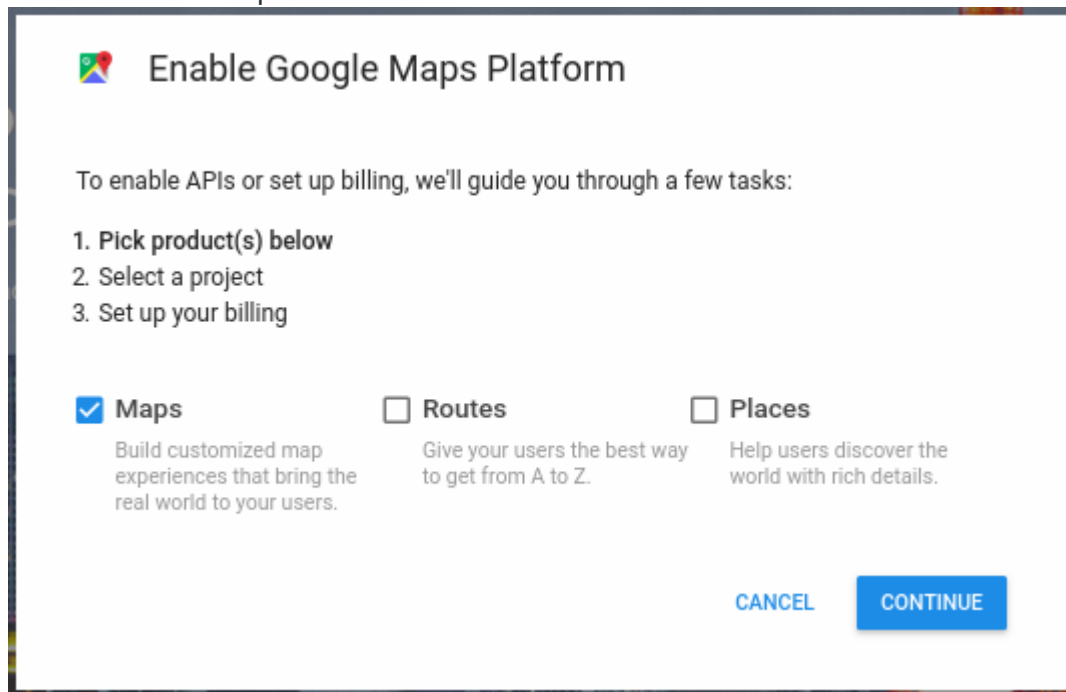
Google Maps in Android

- Maps are of great use and it increases the productivity of an app. Google Maps API allows Android developers to integrate Google Maps in their app. Below is the step-by-step process to integrate Google Maps into Android applications:

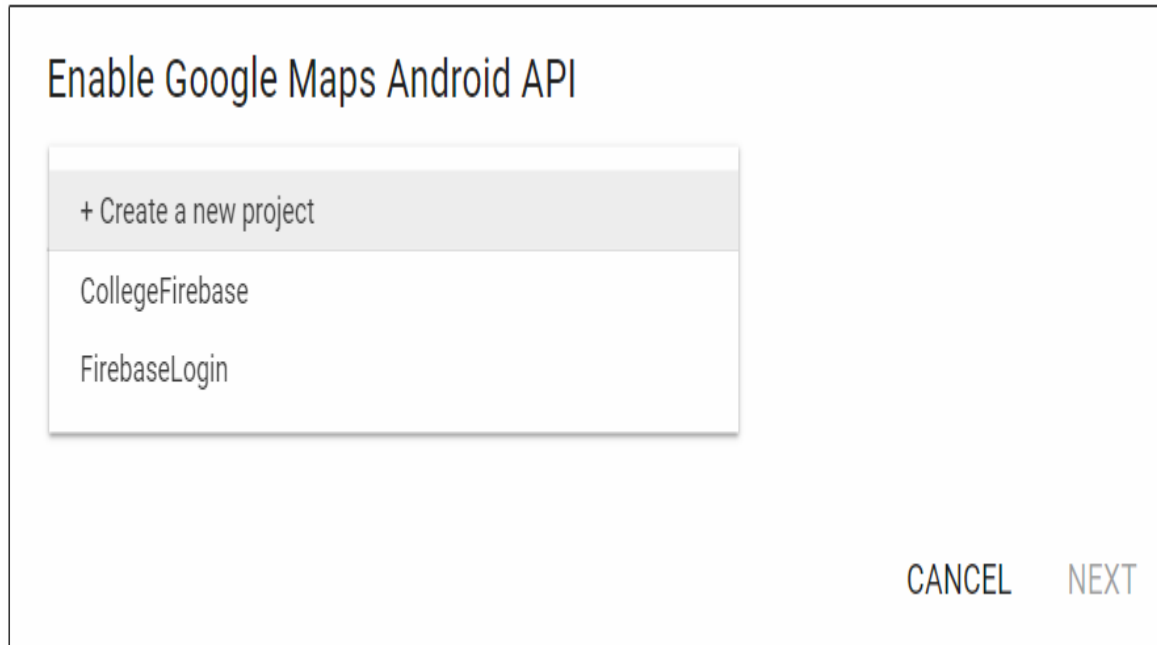
1. Goto <https://developers.google.com/maps/documentation/android-api/signup> and click on “GET STARTED” button as shown in the figure:



1. Now select the Map checkbox and click on the Continue button as shown below:



2. Select a project in which you want to enable Google Map API, and click on Next. A new key will be generated for the chosen project.



3. Skip the Billing Process
4. For integrating Google Map API, your machine's SHA1 (secure Hash Algorithm: to verify the file has been unaltered) certificate is needed. So to find SHA1 certificate, follow below steps:

- Open Command Prompt and go to your Java bin Folder

```
cd C:\Program Files\Java\jdk1.8.0_91\bin
```

- Give the following CMD command for getting Certificate Footprints:
keytool -list -v -keystore "%USERPROFILE%\android\debug.keystore" -alias androiddebugkey -storepass android -keypass android

```
Command Prompt

C:\Program Files\Java\jdk1.8.0_91\bin>keytool -list -v -keystore "%USERPROFILE%\android\debug.keystore" -alias androiddebugkey -storepass android -keypass android
Alias name: androiddebugkey
Creation date: 23 Feb, 2017
Entry type: PrivateKeyEntry
Certificate chain length: 1
Certificate[1]:
Owner: C=US, O=Android, CN=Android Debug
Issuer: C=US, O=Android, CN=Android Debug
Serial number: 1
Valid from: Thu Feb 23 22:15:44 IST 2017 until: Sat Feb 16 22:15:44 IST 2047
Certificate fingerprints:
    MD5: 03:9F:CE:00:DF:EF:67:D0:B9:CB:3C:01:9D:39:17:63
    SHA1: 99:9C:F6:31:EA:0A:7A:CD:94:75:94:62:8F:F3:26:B9:89:5F:B5:5A
    SHA256: 0D:DE:44:C8:D1:5F:8C:E2:C1:37:A1:89:41:EA:27:8F:51:72:93:66:D9:44:C6:0E:0A:08:55:B8:76:72:35:CC
Signature algorithm name: SHA1withRSA
Version: 1

C:\Program Files\Java\jdk1.8.0_91\bin>
```

1. Go to <https://console.developers.google.com/apis/credentials>
2. In the *API keys* section, click on *Pencil* button made on the right of API key that you want to select, for attaching your app with.

API keys			
☐	Name	Creation date ▾	Restrictions
☐	⚠ API key	20 Apr 2018	None
☐	API key 2	17 Apr 2018	Android apps

3. In Application Restrictions, select Android apps

[←](#) API key [↻ REGENERATE KEY](#) [🗑 DELETE](#)

AIzaSyB2eb-oL_otm_HWXTXfWQDCGquFeTxG08

Name
API key

Key restrictions
This key is unrestricted. To prevent unauthorised use and quota theft, restrict your key. [Learn more](#)
⚠ Application restrictions: None ⚠ API restrictions: None

[Application restrictions](#) API restrictions

Application restrictions specify which websites, IP addresses or apps can use this key.

Application restrictions

- ☒ None
- ☐ HTTP referrers (websites)
- ☐ IP addresses (web servers, cron jobs, etc.)
- ☐ Android apps
- ☐ iOS apps

4. Click on Add package name and fingerprint
5. Enter your app's package name and the fingerprint which was found in above steps and click Save button.

Application restrictions

☐ None
☐ HTTP referrers (websites)
☐ IP addresses (web servers, cron jobs, etc.)
☒ Android apps
☐ iOS apps

Restrict usage to your Android apps (Optional)

Add your package name and SHA-1 signing-certificate fingerprint to restrict usage to your Android apps

Get the package name from your AndroidManifest.xml file. Then use the following command to get the fingerprint:

```
$ keytool -list -v -keystore mystore.keystore
```

Package name	SHA-1 certificate fingerprint
com.example.android.maps	99:9C:F6:31:EA:0A:7A:CD:94:75:94:62:8F:F3:26:B9:89:5F:B5:5A
+ Add package name and fingerprint	

Note: It may take up to 5 minutes for settings to take effect.

6. Insert the following in Project ->app ->src ->build.gradle ->dependencies
 compile 'com.google.android.gms:play-services:11.6.0'

activity_maps.xml

```
<fragment xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:map="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/map"
  android:name="com.google.android.gms.maps.SupportMapFragment"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context="example.com.mapexample.MapsActivity" />
```

MapsActivity.java

To get the GoogleMap object in our MapsActivity.java class we need to implement the OnMapReadyCallback interface and override the onMapReady() callback method.

```
package example.com.mapexample;

import android.support.v4.app.FragmentActivity;
import android.os.Bundle;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;

public class MapsActivity extends FragmentActivity implements
OnMapReadyCallback{

    private GoogleMap mMap;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_maps);
        // Obtain the SupportMapFragment and get notified when the map is ready to
be used.

        SupportMapFragment mapFragment = (SupportMapFragment)
getSupportFragmentManager()
                .findFragmentById(R.id.map);
        mapFragment.getMapAsync(this);

    }

    @Override
    public void onMapReady(GoogleMap googleMap) {
        mMap = googleMap;

        // Add a marker in Sydney and move the camera
        LatLng sydney = new LatLng(-34, 151);
        mMap.addMarker(new MarkerOptions().position(sydney).title("Marker in
Sydney"));
        mMap.moveCamera(CameraUpdateFactory.newLatLng(sydney));
    }
}
```

```
}  
}
```

Required Permission

Add the following user-permission in AndroidManifest.xml file.

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"  
</>  
<uses-permission android:name="android.permission.INTERNET" />  
AndroidManifest.xml  
<?xml version="1.0" encoding="utf-8"?>  
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    package="example.com.mapexample">  
    <!--  
        The ACCESS_COARSE/FINE_LOCATION permissions are not required to use  
        Google Maps Android API v2, but you must specify either coarse or fine  
        location permissions for the 'MyLocation' functionality.  
    -->  
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
    <uses-permission  
android:name="android.permission.ACCESS_COARSE_LOCATION" />  
    <uses-permission android:name="android.permission.INTERNET" />  
  
    <application  
        android:allowBackup="true"  
        android:icon="@mipmap/ic_launcher"  
        android:label="@string/app_name"  
        android:roundIcon="@mipmap/ic_launcher_round"  
        android:supportsRtl="true"  
        android:theme="@style/AppTheme">  
  
        <meta-data  
            android:name="com.google.android.geo.API_KEY"  
            android:value="@string/google_maps_key" />  
  
        <activity  
            android:name=".MapsActivity"  
            android:label="@string/title_activity_maps">  
            <intent-filter>  
                <action android:name="android.intent.action.MAIN" />
```

```

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
</application>

</manifest>

```

build.gradle

Add the following dependencies in build.gradle file.

```

dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar'])
    implementation 'com.android.support:appcompat-v7:26.1.0'
    implementation 'com.google.android.gms:play-services-maps:11.8.0'
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'com.android.support.test:runner:1.0.1'
    androidTestImplementation 'com.android.support.test.espresso:espresso-
core:3.0.1'
}

```

Android - Location Based Services

A location-based service (LBS) is a software service for mobile device applications that requires knowledge about where the mobile device is geographically located. The application collects geodata, which is data gathered in Real Time using one or more location tracking technologies.

Location-based services integrate data from various resources, including Global Positioning System (GPS) satellites, cellular tower pings and short-range positioning beacons, to provide services based on the user's geographical location. Although location-based technologies have been commercially available for almost two decades, the applications and services that use geodata have recently become mainstream, thanks to the widespread use of Android and Apple smartphones and tablets.

The Location Object

The Location object represents a geographic location which can consist of a latitude, longitude, time stamp, and other information such as bearing, altitude and velocity. There are following important methods which you can use with Location object to get location specific information –

Sr.No.	Method & Description
--------	----------------------

1. float distanceTo(Location dest): Returns the approximate distance in meters between this location and the given location.
2. float getAccuracy(): Get the estimated accuracy of this location, in meters.
3. double getAltitude(): Get the altitude if available, in meters above sea level.
4. float getBearing(): Get the bearing, in degrees.
5. double getLatitude(): Get the latitude, in degrees.
6. double getLongitude(): Get the longitude, in degrees.
7. float getSpeed(): Get the speed if it is available, in meters/second over ground.
8. boolean hasAccuracy(): True if this location has an accuracy.
9. boolean hasAltitude(): True if this location has an altitude.
10. boolean hasBearing(): True if this location has a bearing.
11. boolean hasSpeed(): True if this location has a speed.
12. void reset(): Clears the contents of the location.
13. void setAccuracy(float accuracy): Set the estimated accuracy of this location, meters.
14. void setAltitude(double altitude): Set the altitude, in meters above sea level.
15. void setBearing(float bearing): Set the bearing, in degrees.
16. void setLatitude(double latitude): Set the latitude, in degrees.
17. void setLongitude(double longitude): Set the longitude, in degrees.
18. void setSpeed(float speed): Set the speed, in meters/second over ground.
19. String toString(): Returns a string containing a concise, human-readable description of this object.

Get the Current Location

To get the current location, create a location client which is LocationClient object, connect it to Location Services using connect() method, and then call its getLastLocation() method. This method returns the most recent location in the form of Location object that contains latitude and longitude coordinates and other information as explained above. To have location based functionality in your activity, you will have to implement two interfaces –

GooglePlayServicesClient.ConnectionCallbacks

GooglePlayServicesClient.OnConnectionFailedListener

These interfaces provide following important callback methods, which you need to implement in your activity class –

Sr.No. Callback Methods & Description

1. abstract void onConnected(Bundle connectionHint): This callback method is called when location service is connected to the location client successfully. You will use connect() method to connect to the location client.
2. abstract void onDisconnected(): This callback method is called when the client is disconnected. You will use disconnect() method to disconnect from the location client.

3. `abstract void onConnectionFailed(ConnectionResult result)`: This callback method is called when there was an error connecting the client to the service.

You should create the location client in `onCreate()` method of your activity class, then connect it in `onStart()`, so that Location Services maintains the current location while your activity is fully visible. You should disconnect the client in `onStop()` method, so that when your app is not visible, Location Services is not maintaining the current location. This helps in saving battery power up-to a large extent.

Get the Updated Location

If you are willing to have location updates, then apart from above mentioned interfaces, you will need to implement `LocationListener` interface as well. This interface provide following callback method, which you need to implement in your activity class –

Sr.No. Callback Method & Description

1

`abstract void onLocationChanged(Location location)`

This callback method is used for receiving notifications from the `LocationClient` when the location has changed.

Location Quality of Service

The `LocationRequest` object is used to request a quality of service (QoS) for location updates from the `LocationClient`. There are following useful setter methods which you can use to handle QoS. There are equivalent getter methods available which you can check in Android official documentation.

Sr.No. Method & Description

1. `setExpirationDuration(long millis)`: Set the duration of this request, in milliseconds.
2. `setExpirationTime(long millis)`: t the request expiration time, in millisecond since boot.
3. `setFastestInterval(long millis)`: Explicitly set the fastest interval for location updates, in milliseconds.
4. `setInterval(long millis)`: Set the desired interval for active location updates, in milliseconds.
5. `setNumUpdates(int numUpdates)`: Set the number of location updates.

6. `setPriority(int priority)`: Set the priority of the request.

Now for example, if your application wants high accuracy location it should create a location request with `setPriority(int)` set to `PRIORITY_HIGH_ACCURACY` and `setInterval(long)` to 5 seconds. You can also use bigger interval and/or other priorities like `PRIORITY_LOW_POWER` for to request "city" level accuracy or `PRIORITY_BALANCED_POWER_ACCURACY` for "block" level accuracy.

Example

Following example shows you in practical how to to use Location Services in your app to get the current location and its equivalent addresses etc.

To experiment with this example, you will need actual Mobile device equipped with latest Android OS, otherwise you will have to struggle with emulator which may not work.

Create Android Application

Step	Description
1	You will use Android studio IDE to create an Android application and name it as Tutorialspoint under a package <code>com.example.tutorialspoint7.myapplication</code> .
2	add <code>src/GPSTracker.java</code> file and add required code.
3	Modify <code>src/MainActivity.java</code> file and add required code as shown below to take care of getting current location and its equivalent address.
4	Modify layout XML file <code>res/layout/activity_main.xml</code> to add all GUI components which include three buttons and two text views to show location/address.
5	Modify <code>res/values/strings.xml</code> to define required constant values
6	Modify <code>AndroidManifest.xml</code> as shown below
7	Run the application to launch Android emulator and verify the result of the changes done in the application.

Following is the content of the modified main activity file `MainActivity.java`.

```
package com.example.maps.lbs;
```

```
import android.Manifest;
```

```
import android.app.Activity;
```

```
import android.os.Bundle;
```

```
import android.support.v4.app.ActivityCompat;
```

```
import android.test.mock.MockPackageManager;
```

```
import android.view.View;
```

```
import android.widget.Button;
```

```
import android.widget.Toast;
```

```
public class MainActivity extends Activity {
```

```

Button btnShowLocation;
private static final int REQUEST_CODE_PERMISSION = 2;
String mPermission = Manifest.permission.ACCESS_FINE_LOCATION;

// GPSTracker class
GPSTracker gps;

@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    try {
        if (ActivityCompat.checkSelfPermission(this, mPermission)
            != MockPackageManager.PERMISSION_GRANTED) {

            ActivityCompat.requestPermissions(this, new String[]{mPermission},
                REQUEST_CODE_PERMISSION);

            // If any permission above not allowed by user, this condition will
            // execute every time, else your else part will work
        }
    } catch (Exception e) {
        e.printStackTrace();
    }

    btnShowLocation = (Button) findViewById(R.id.button);

    // show location button click event
    btnShowLocation.setOnClickListener(new View.OnClickListener() {

        @Override
        public void onClick(View arg0) {
            // create class object
            gps = new GPSTracker(MainActivity.this);

            // check if GPS enabled
            if(gps.canGetLocation()){

                double latitude = gps.getLatitude();

```

```

        double longitude = gps.getLongitude();

        // \n is for new line
        Toast.makeText(getApplicationContext(), "Your Location is - \nLat: "
            + latitude + "\nLong: " + longitude, Toast.LENGTH_LONG).show();
    }else{
        // can't get location
        // GPS or Network is not enabled
        // Ask user to enable GPS/network in settings
        gps.showSettingsAlert();
    }

    }
    });
}
}

```

Following is the content of the modified main activity file GPSTracker.java.

```

package com.example.maps.lbs;

import android.app.AlertDialog;
import android.app.Service;
import android.content.Context;
import android.content.DialogInterface;
import android.content.Intent;
import android.location.Location;
import android.location.LocationListener;
import android.location.LocationManager;
import android.os.Bundle;
import android.os.IBinder;
import android.provider.Settings;
import android.util.Log;

public class GPSTracker extends Service implements LocationListener {

    private final Context mContext;

    // flag for GPS status
    boolean isGPSEnabled = false;

```

```

// flag for network status
boolean isNetworkEnabled = false;

// flag for GPS status
boolean canGetLocation = false;

Location location; // location
double latitude; // latitude
double longitude; // longitude

// The minimum distance to change Updates in meters
private static final long MIN_DISTANCE_CHANGE_FOR_UPDATES = 10; // 10 meters

// The minimum time between updates in milliseconds
private static final long MIN_TIME_BW_UPDATES = 1000 * 60 * 1; // 1 minute

// Declaring a Location Manager
protected LocationManager locationManager;

public GPSTracker(Context context) {
    this.mContext = context;
    getLocation();
}

public Location getLocation() {
    try {
        locationManager = (LocationManager)
mContext.getSystemService(LOCATION_SERVICE);

        // getting GPS status
        isGPSEnabled = locationManager.isProviderEnabled(LocationManager.GPS_PROVIDER);

        // getting network status
        isNetworkEnabled = locationManager
            .isProviderEnabled(LocationManager.NETWORK_PROVIDER);

        if (!isGPSEnabled && !isNetworkEnabled) {
            // no network provider is enabled
        } else {
            this.canGetLocation = true;
            // First get location from Network Provider

```

```

if (isNetworkEnabled) {
    locationManager.requestLocationUpdates(
        LocationManager.NETWORK_PROVIDER,
        MIN_TIME_BW_UPDATES,
        MIN_DISTANCE_CHANGE_FOR_UPDATES, this);

    Log.d("Network", "Network");
    if (locationManager != null) {
        location = locationManager
            .getLastKnownLocation(LocationManager.NETWORK_PROVIDER);

        if (location != null) {
            latitude = location.getLatitude();
            longitude = location.getLongitude();
        }
    }
}

// if GPS Enabled get lat/long using GPS Services
if (isGPSEnabled) {
    if (location == null) {
        locationManager.requestLocationUpdates(
            LocationManager.GPS_PROVIDER,
            MIN_TIME_BW_UPDATES,
            MIN_DISTANCE_CHANGE_FOR_UPDATES, this);

        Log.d("GPS Enabled", "GPS Enabled");
        if (locationManager != null) {
            location = locationManager
                .getLastKnownLocation(LocationManager.GPS_PROVIDER);

            if (location != null) {
                latitude = location.getLatitude();
                longitude = location.getLongitude();
            }
        }
    }
}

} catch (Exception e) {

```

```

        e.printStackTrace();
    }

    return location;
}

/**
 * Stop using GPS listener
 * Calling this function will stop using GPS in your app
 */

public void stopUsingGPS(){
    if(locationManager != null){
        locationManager.removeUpdates(GPSTracker.this);
    }
}

/**
 * Function to get latitude
 */

public double getLatitude(){
    if(location != null){
        latitude = location.getLatitude();
    }

    // return latitude
    return latitude;
}

/**
 * Function to get longitude
 */

public double getLongitude(){
    if(location != null){
        longitude = location.getLongitude();
    }

    // return longitude
    return longitude;
}

```

```

}

/**
 * Function to check GPS/wifi enabled
 * @return boolean
 * */

public boolean canGetLocation() {
    return this.canGetLocation;
}

/**
 * Function to show settings alert dialog
 * On pressing Settings button will launch Settings Options
 * */

public void showSettingsAlert(){
    AlertDialog.Builder alertDialog = new AlertDialog.Builder(mContext);

    // Setting Dialog Title
    alertDialog.setTitle("GPS is settings");

    // Setting Dialog Message
    alertDialog.setMessage("GPS is not enabled. Do you want to go to settings menu?");

    // On pressing Settings button
    alertDialog.setPositiveButton("Settings", new DialogInterface.OnClickListener() {
        public void onClick(DialogInterface dialog,int which) {
            Intent intent = new Intent(Settings.ACTION_LOCATION_SOURCE_SETTINGS);
            mContext.startActivity(intent);
        }
    });

    // on pressing cancel button
    alertDialog.setNegativeButton("Cancel", new DialogInterface.OnClickListener() {
        public void onClick(DialogInterface dialog, int which) {
            dialog.cancel();
        }
    });

    // Showing Alert Message

```

```
    alertDialog.show();  
}
```

```
@Override  
public void onLocationChanged(Location location) {  
}
```

```
@Override  
public void onProviderDisabled(String provider) {  
}
```

```
@Override  
public void onProviderEnabled(String provider) {  
}
```

```
@Override  
public void onStatusChanged(String provider, int status, Bundle extras) {  
}
```

```
@Override  
public IBinder onBind(Intent arg0) {  
    return null;  
}  
}
```

Following will be the content of res/layout/activity_main.xml file –

```
<?xml version = "1.0" encoding = "utf-8"?>  
<LinearLayout xmlns:android = "http://schemas.android.com/apk/res/android"  
    android:layout_width = "fill_parent"  
    android:layout_height = "fill_parent"  
    android:orientation = "vertical" >
```

```
<Button  
    android:id = "@+id/button"  
    android:layout_width = "fill_parent"  
    android:layout_height = "wrap_content"  
    android:text = "getLocation"/>
```

```
</LinearLayout>
```

Following will be the content of res/values/strings.xml to define two new constants –

```
<?xml version = "1.0" encoding = "utf-8"?>
<resources>
  <string name = "app_name">Tutorialspoint</string>
</resources>
```

Following is the default content of AndroidManifest.xml –

```
<?xml version = "1.0" encoding = "utf-8"?>
<manifest xmlns:android = "http://schemas.android.com/apk/res/android"
  package = "com.example.tutorialspoint7.myapplication">
  <uses-permission android:name = "android.permission.ACCESS_FINE_LOCATION" />
  <uses-permission android:name = "android.permission.INTERNET" />
  <application
    android:allowBackup = "true"
    android:icon = "@mipmap/ic_launcher"
    android:label = "@string/app_name"
    android:supportsRtl = "true"
    android:theme = "@style/AppTheme">

    <activity android:name = ".MainActivity">
      <intent-filter>
        <action android:name = "android.intent.action.MAIN" />

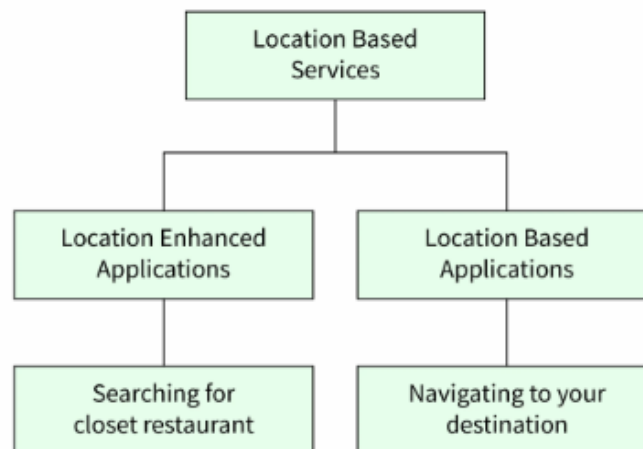
        <category android:name = "android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>

</manifest>
```

Location-Based Services(LBS) in Android provide valuable functionality by leveraging a device's GPS, Wi-Fi, or cellular network to deliver location-specific information and services. These services enable applications to determine the user's current location, track movement and offer personalized experiences. LBS in Android facilitates features such as mapping, navigation, geofencing, location-aware advertisements, and social check-ins. Developers can access location data through the Android Location API, which offers various location providers and methods for obtaining accurate and up-to-date location information. With LBS, Android apps can deliver tailored content and services based on the user's geographic position, enhancing user experiences and enabling location-aware functionality.

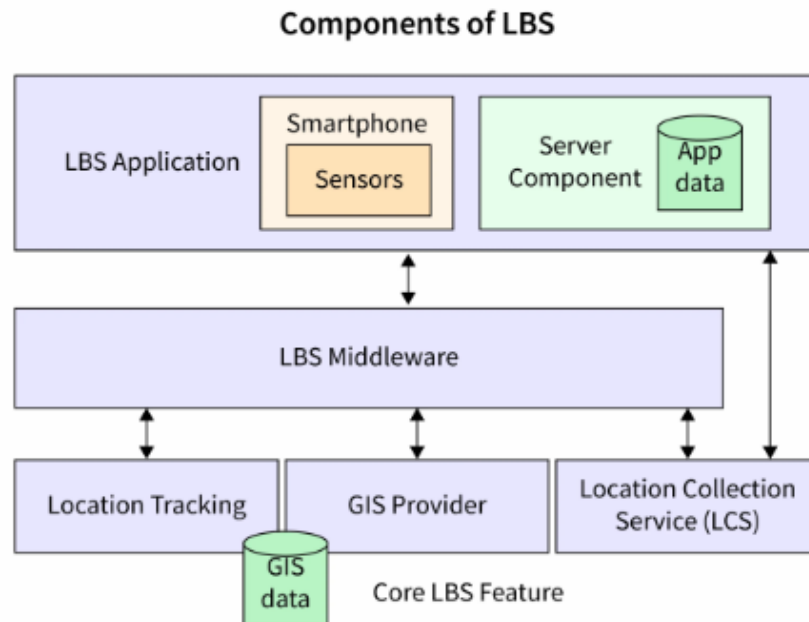


What are Android Location-Based Services?



Android Location-Based Services (LBS) refer to the functionalities and capabilities provided by the Android operating system that leverage a device's location data to deliver location-specific information and services. LBS in Android enables developers to create applications that can determine the user's current geographic position, track movement, and offer personalized experiences based on their location.

Components of Location-Based Services in Android



Location-Based Services (LBS) in Android rely on several key components to provide accurate and up-to-date location information. These components include the `LocationManager` class, the `LocationListener` interface, the `LocationProvider`, and the `Location` class. Let's explore each of these components in detail.

LocationManager Class

The `LocationManager` class in Android is a central component responsible for interacting with the system's location services and managing location updates. It acts as a bridge between the application and the underlying hardware and services that provide location information. Developers use the `LocationManager` class to request location updates, retrieve the last known location, and manage location providers. To obtain an instance of the `LocationManager` class, developers can use the `getSystemService()` method, passing the `LOCATION_SERVICE` constant. This allows the application to access the system's location services and perform location-related operations.

LocationListener Interface

The `LocationListener` interface is used to receive location updates from the `LocationManager` class. Developers implement this interface and override its methods to handle location updates and respond to changes in the user's position. The `LocationListener` interface defines three methods:

onLocationChanged(): This method is called when a new location update is available. Developers can retrieve the updated location from the Location parameter passed to this method and perform any necessary actions based on the new location data. For example, an application might update a map with the user's current location or trigger location-based notifications.

onStatusChanged(): This method is invoked when the status of the location provider changes. It provides information about the availability, enabling, or disabling of a location provider. Developers can use this method to handle changes in the availability of location providers and adapt their application's behavior accordingly. For example, if the GPS provider becomes temporarily unavailable, the application might switch to using network-based location updates.

onProviderEnabled() and onProviderDisabled(): These methods are called when a location provider is enabled or disabled, respectively. Developers can use these methods to handle changes in the availability of location providers and inform the user about the status of location services. For example, an application might display a message when GPS is disabled, prompting the user to enable it for better location accuracy.

LocationProvider

A LocationProvider represents a source of location data, such as GPS, network-based location, or other custom location sources.

Android provides several built-in location providers:

GPS_PROVIDER: Uses the device's Global Positioning System (GPS) to provide accurate location information. It offers high accuracy but may consume more power and require a clear line of sight to GPS satellites.

NETWORK_PROVIDER: Utilizes Wi-Fi and cellular network signals to determine the user's location. It offers location updates with lower power consumption but may be less accurate than GPS. Network-based location can be particularly useful in urban areas with dense Wi-Fi networks.

PASSIVE_PROVIDER: Allows the application to receive location updates passively, without directly requesting them. This provider listens for location updates generated by other applications or services. It is useful when an application needs location updates but does not require controlling the frequency or source of those updates.

Location Class

The Location class in Android represents a geographic location. It encapsulates the latitude, longitude, altitude, and other optional information, such as speed and bearing, associated with a particular location. Developers can obtain a Location object either from location updates received via the LocationListener interface or by querying the LocationManager for the last known location using the getLastKnownLocation() method. The Location class

provides methods to access and manipulate location data, such as getting latitude and longitude coordinates, calculating the distance between locations, and retrieving additional information like altitude and speed.

Location Object

The Location object in Android is a key component of Location-Based Services (LBS) that represents a geographic location. It encapsulates information such as latitude, longitude, altitude, and other optional attributes associated with a particular location. The Location object provides methods and properties to access and manipulate location data, making it an essential class for handling location-related functionality in Android applications.

Location Quality of Service

Location Quality of Service (QoS) is a crucial aspect of Location-Based Services (LBS) in Android. It refers to the level of accuracy, reliability, and availability of location information provided to applications. Developers need to consider QoS factors when designing location-aware applications to ensure a satisfactory user experience and effective functionality. Android offers various mechanisms and parameters to assess and manage the quality of location services.

Accuracy: Accuracy is a fundamental aspect of location services. It refers to the degree of closeness between the reported location and the user's actual position. Android provides a measure of accuracy through the Location object's `getAccuracy()` method. Accuracy is typically represented as a radius within which the actual location is likely to exist. Smaller accuracy values indicate higher precision, while larger values indicate a broader range of possible locations. Developers can utilize this information to make decisions based on the reliability of the location fix and ensure accurate positioning in their applications. For example, applications requiring precise navigation or proximity-based functionality may need higher accuracy levels.

Power Consumption: Location services, particularly those that rely on GPS, can consume significant amounts of power. To balance accuracy and power efficiency, Android offers different location providers with varying power consumption profiles. Developers can choose between `GPS_PROVIDER`, `NETWORK_PROVIDER`, and `PASSIVE_PROVIDER` based on their application's requirements. `GPS_PROVIDER` provides high accuracy but consumes more power, while `NETWORK_PROVIDER` utilizes Wi-Fi and cellular signals with lower power consumption but potentially lower accuracy. The choice of location provider can significantly impact the battery life of the device and should be considered when designing location-based functionality. Developers should strike a balance between accuracy and power consumption to ensure an optimal user experience.

Availability: Location services may not always be available due to various factors such as device hardware limitations, user settings, or environmental conditions. For example, indoor environments or areas with poor GPS reception may result in limited availability of GPS-based location updates. Android provides mechanisms to determine the availability of location providers. The `LocationManager` class offers methods such as `isProviderEnabled()` to check if a specific provider is enabled on the device. By considering the availability of location services, developers can provide appropriate fallback mechanisms or user notifications when location services are not accessible. For example, an application might switch to network-based location updates if GPS is unavailable or prompt the user to enable location services if they are disabled.

Time-to-First-Fix: Time-to-First-Fix (TTFF) refers to the duration it takes for a location provider to acquire the first location fix after being enabled. This is particularly relevant for GPS-based location providers, as they need time to establish a connection with satellites and obtain accurate position information. Android provides the `getElapsedRealtimeNanos()` method in the `Location` object, which can be used to measure the time it takes to acquire the first fix. Developers can monitor the TTFF and manage user expectations accordingly, such as displaying loading indicators or progress bars during the acquisition process. Users may experience delays in receiving accurate location updates, especially when using GPS, and being transparent about the acquisition process can enhance the user experience.

Location Update Frequency: The frequency at which location updates are received can impact the user experience and power consumption. Android allows developers to specify the time interval and distance threshold for location updates through the `LocationManager` class. By selecting appropriate values for these parameters, developers can balance the need for real-time location updates with power efficiency. For example, an application that requires frequent location updates for real-time tracking may opt for shorter intervals, while an application focused on periodic updates for less time-sensitive information may choose longer intervals to conserve power. Adjusting the update frequency based on the application's requirements can optimize the user experience and power consumption.

Geocoders

Geocoders in Android are essential components of Location-Based Services (LBS) that facilitate the conversion of textual addresses or place names into geographic coordinates (latitude and longitude) and vice versa. Android provides the `Geocoder` class as an API to perform geocoding and reverse geocoding operations, enabling developers to incorporate geospatial functionality into their applications.

When working with Geocoders in Android, developers should handle geocoding results effectively. Multiple `Address` objects can be returned for a single geocoding or reverse geocoding query. Developers should consider the possibility of multiple matches and

determine the most relevant or accurate address based on their application's requirements. It's important to handle potential errors or empty result sets appropriately and inform users accordingly. Geocoding results may vary in accuracy and availability, depending on factors such as the underlying mapping service and the quality of the input.

Implementation of Location-Based Services in Android

Implementing Location-Based Services (LBS) in Android involves leveraging the platform's built-in location APIs and integrating them into your application. Here are the key steps to implement LBS in Android:

Request Location Permissions: First, ensure that your application has the necessary permissions to access the device's location. In the `AndroidManifest.xml` file, add the required permissions such as `ACCESS_FINE_LOCATION` or `ACCESS_COARSE_LOCATION`. Additionally, starting from Android 10 (API level 29), you may need to request runtime permissions from the user explicitly.

Use the LocationManager Class: The `LocationManager` class is the central component for accessing location services in Android. Obtain an instance of the `LocationManager` using `getSystemService()` and the `LOCATION_SERVICE` constant.

Select Location Provider: Choose the appropriate location provider based on your application's requirements. Android provides different location providers such as `GPS_PROVIDER`, `NETWORK_PROVIDER`, and `PASSIVE_PROVIDER`. `GPS_PROVIDER` offers high accuracy but consumes more power, while `NETWORK_PROVIDER` utilizes Wi-Fi and cellular signals with lower power consumption but potentially lower accuracy. Select the provider that suits your needs.

Register a LocationListener: Implement the `LocationListener` interface and register it with the `LocationManager` using the `requestLocationUpdates()` method. This enables your application to receive periodic location updates. Implement the necessary callback methods, such as `onLocationChanged()`, to handle location updates and process the received location data.

Handle Location Updates: In the `onLocationChanged()` method, extract the location data from the `Location` object provided as a parameter. You can access latitude, longitude, altitude, speed, and other location attributes. Process this data according to your application's requirements, such as updating a map, triggering notifications, or calculating distances.

Geocoding and Reverse Geocoding: To convert textual addresses or place names into geographic coordinates (geocoding) or vice versa (reverse geocoding), use the `Geocoder`

class. Instantiate a Geocoder object and call methods like `getFromLocationName()` or `getFromLocation()` to retrieve Address objects representing location information.

Handle Errors and Edge Cases: Consider potential errors and edge cases in location services. For example, handle scenarios where location services are disabled or unavailable, where permissions are not granted, or where no location data is available. Provide appropriate error messages or fallback mechanisms to ensure a smooth user experience.

Optimize Power Consumption: To optimize power consumption, consider factors like location update frequency and location provider selection. Determine the appropriate update interval based on your application's needs and the desired balance between real-time updates and power efficiency. Use lower power-consuming location providers, such as `NETWORK_PROVIDER`, when high accuracy is not crucial.

Enhance User Experience: Leverage location data to enhance the user experience. For example, integrate maps to display the user's location, provide directions, or highlight nearby points of interest. Implement geofencing to trigger specific actions when the user enters or exits predefined geographical areas. Utilise location-based advertising or personalised recommendations based on the user's current location.